

Busca de hiperparâmetros: Grid Search

Parâmetro × hiperparâmetro, GridSearchCV e o custo da busca exaustiva (Aula 08)

De onde viemos: já sabemos MEDIR, agora vamos ESCOLHER

Na Aula 07 aprendemos a **validação cruzada**: em vez de um corte, várias dobras, e o resultado é a **média $\pm$ desvio**. Hoje usamos essa régua para escolher os ajustes do modelo.

- **O problema (recap)** — a CV dá uma estimativa estável e honesta — mas ainda não dissemos QUAIS valores (C, profundidade, k) usar no modelo.
- **A ideia de hoje** — testar várias combinações de hiperparâmetros numa GRADE e deixar a validação cruzada apontar a melhor.
- **E mais** — contar o custo dessa busca (a grade explode) e confirmar o campeão uma única vez no teste — sem vazar.

Objetivo da aula

Ajustar hiperparâmetros por **busca em grade** (Grid Search), avaliando cada combinação na **validação cruzada**, sem tocar no teste.

O CAMINHO

- **Parâmetro × hiperparâmetro** — o que o modelo aprende vs. o que você define.
- **GridSearchCV** — testar toda a grade, com CV, e ler os resultados.
- **O custo** — por que a grade explode — e confirmar no teste.

A ANALOGIA DE HOJE

- **A receita do bolo** — a grade de tempo × temperatura: você testa as combinações e acha a que assa melhor.
- Vale para qualquer modelo e seus hiperparâmetros.

Da Aula 07 (validação cruzada) para a Aula 08: usar a mesma validação para ESCOLHER os hiperparâmetros.

PARTE 1

Parâmetro e hiperparâmetro

O que o modelo aprende sozinho e o que você precisa definir.

1

Parâmetro × hiperparâmetro

Parâmetro

- **O modelo aprende** — durante o fit, a partir dos dados.
- **Exemplos** — os pesos de uma regressão, os cortes de uma árvore.
- **Você não define** — sai do treino, não da sua mão.

Hiperparâmetro

- **Você define antes** — configura o modelo antes de treinar.
- **Exemplos** — C da regularização, profundidade da árvore, k do k-NN.
- **É o que ajustamos hoje** — a busca em grade escolhe o melhor valor.

Analogia do bolo: os parâmetros são como a massa cresce no forno (o processo); os hiperparâmetros são o tempo e a temperatura que VOCÊ escolhe.

Por que ajustar — e a regra de ouro

Bons hiperparâmetros melhoram o modelo. Mas escolhê-los olhando o teste é **vazamento**: ajuste na **validação** e confirme uma vez no **teste**.

- **O ganho** — o mesmo algoritmo pode ir de medíocre a ótimo só com o ajuste certo.
- **O risco** — testar muitas opções no teste 'gasta' a imparcialidade dele.
- **O fluxo** — treino aprende; validação escolhe; teste confirma — uma vez só.

É a mesma regra de ouro da Aula 07: quem escolhe não mede. A busca vive na validação, nunca no teste.

PARTE 2

GridSearchCV

Testar todas as combinações de uma grade, com validação cruzada.

2

A busca exaustiva

Você define uma **grade de valores**; o GridSearchCV treina e valida **cada combinação** (com CV) e escolhe a de melhor média.

- **Você define a grade** — as opções de cada hiperparâmetro (ex.: $C \in \{0,1; 1; 10\}$).
- **Ele varre tudo** — cada combinação vira um modelo, avaliado por validação cruzada.
- **Ele escolhe o topo** — a combinação com a melhor média de CV vence — é o `best_params_`.

'Exaustiva' porque não deixa combinação de fora: é a grade inteira, sem atalhos.

A receita do bolo: uma grade de verdade

Duas coisas você controla no bolo: o **tempo** e a **temperatura**. A grade testa todas as duplas e mede qual assa melhor.

nota do bolo (0–10)	160°C	180°C	200°C
30 min	6,0	7,0	6,5
40 min	7,5	9,0	7,0
50 min	6,5	7,5	5,5

Melhor dupla: 40 min × 180°C (nota 9,0). É exatamente o que o GridSearchCV faz com C e class_weight em vez de tempo e temperatura.

GridSearchCV na prática

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold

param_grid = {
    "clf__C": [0.1, 1, 10],          # prefixo do passo do Pipeline
    "clf__class_weight": [None, "balanced"],
}
cv = StratifiedKFold(5, shuffle=True, random_state=0)
busca = GridSearchCV(pipe, param_grid, scoring="average_precision", cv=cv)
busca.fit(X_tr, y_tr)  # roda a CV para cada combinacao
```

A busca roda sobre o Pipeline: o pré-processamento se refaz em cada dobra, sem vaziar (recap da Aula 07).

Dois cuidados: o prefixo e o scoring

- **O prefixo do passo** — na grade, use `nome_do_passo__parametro` (ex.: `clf__C`) para mirar o passo certo do Pipeline.
- **O scoring certo** — escolha a métrica que importa (F1, `average_precision`, `roc_auc`) — não a acurácia com classe rara.
- **cv estratificado** — em classificação, use `StratifiedKFold` para preservar a classe rara em cada dobra.

Errar o prefixo ou o scoring faz a busca 'otimizar' a coisa errada — a grade fica certa, mas a escolha, não.

PARTE 3

O custo da busca

Por que a grade explode — o exemplo na mão.

3

A conta do grid search

NÚMERO DE TREINOS

$(\text{opções do eixo 1}) \times (\text{opções do eixo 2}) \times \dots \times (\text{nº de dobras da CV})$

No nosso exemplo: **3 valores de C** $\times$ **2 de class_weight** $\times$ **5 dobras** = **30 treinos**.

- **Cada eixo novo multiplica** — acrescentar um hiperparâmetro com 4 opções multiplica tudo por 4.
- **A consequência** — grades grandes ficam caríssimas — o que motiva a busca aleatória (Aula 09).

6 combinações $\times$ 5 dobras = 30 modelos treinados só para uma grade pequena. É o preço da busca exaustiva.

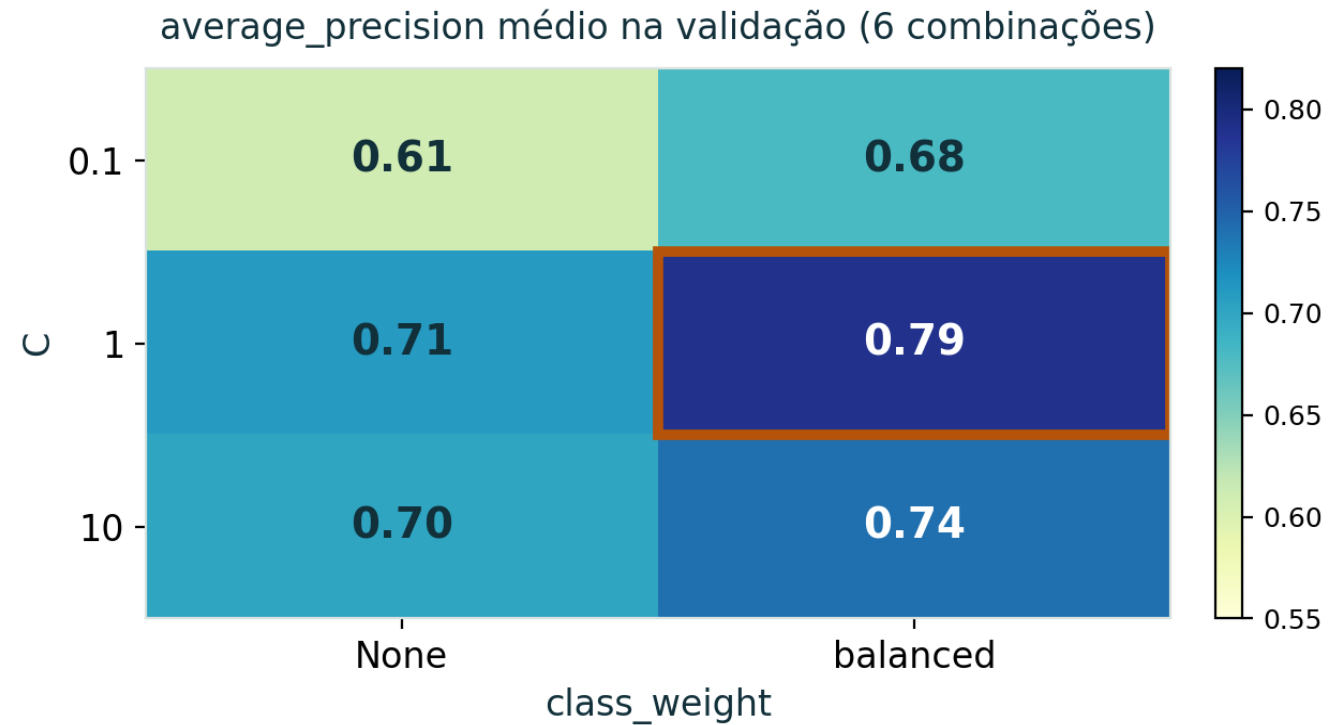
O exemplo na mão: as 6 combinações

A grade tem **3 × 2 = 6 combinações**; cada uma é avaliada em **5 dobras** — a nota abaixo é a MÉDIA dessas 5.

C	class_weight	média CV (AP)	treinos
0,1	None	0,61	5
0,1	balanced	0,68	5
1	None	0,71	5
1	balanced	0,79 ← melhor	5
10	None	0,70	5
10	balanced	0,74	5

Soma dos treinos: 6 × 5 = 30. best_params_ = {C: 1, class_weight: balanced}; best_score_ = 0,79 (validação, não teste).

A grade, vista de cima



Cada célula é a média de CV de uma combinação. O quadro âmbar marca a vencedora (C=1, balanced). Vizinhas parecidas → há um platô.

PARTE 4

Lendo os resultados

O que a busca guarda — e o que é honesto reportar.



O que o objeto guarda

```
busca.best_params_      # a melhor combinacao encontrada
busca.best_score_       # media de CV dela (validacao, NAO teste)
busca.best_estimator_   # o Pipeline ja treinado com o melhor

import pandas as pd
pd.DataFrame(busca.cv_results_)  # a tabela completa
```

- **cv_results_** — tem a média e o desvio de cada combinação — ordene para ver o pódio.
- **best_estimator_** — já vem treinado; é o modelo que você levará ao teste.

O objeto guarda tudo: quem venceu, por quanto, e a tabela inteira para você inspecionar o platô.

Cuidado: `best_score_` vem de dentro

O **`best_score_`** é a validação, não o teste. Como é o **melhor de muitas medições**, tende a ser um pouco otimista.

- **A maldição do vencedor** — vista na Aula 07 — o topo de muitas dobras ruidosas infla por sorte.
- **Olhe o platô** — se várias combinações empatam dentro do desvio, prefira a mais simples.
- **A palavra final** — é o teste, tocado uma única vez com o `best_estimator_`.

`best_score_` ajuda a ESCOLHER, mas não é o número que se reporta — esse vem do teste.

PARTE 5

Do resultado ao teste

Confirmar o campeão uma única vez.

5

best_estimator_ e a palavra final

O **best_estimator_** já está treinado com a melhor combinação. Confirme-o **uma vez no teste** — esse é o número honesto.

- **Cenário saudável** — o teste fica próximo do best_score_ da validação.
- **Sinal de alerta** — teste bem abaixo da validação sugere sobreajuste da busca (grade grande demais).
- **Reporte** — o desempenho do teste, não o best_score_ da busca.

O ciclo fecha aqui: treino aprendeu, validação escolheu, teste confirmou — uma vez só.

Tabela de decisão: montando a grade

Situação	Escolha	Por quê
Métrica com classe rara	scoring = F1 / average_precision	a acurácia engana com desbalanceio
Classificação (padrão)	cv = StratifiedKfold	preserva a proporção das classes na dobra
Pré-processamento no meio	GridSearchCV sobre o Pipeline	o prep se refaz em cada dobra, sem vaziar
Muitos hiperparâmetros	comece pequeno, depois refine	a grade cheia explode em nº de treinos
Reportar desempenho	teste com best_estimator_	best_score_ é otimista (vem de dentro)

A grade é uma decisão de projeto: métrica certa, CV certo, dentro do Pipeline — e começa enxuto.

PARTE 6

Boas práticas

Como definir a grade e o que evitar.

6

Erros comuns e boas práticas

Erros comuns

- Ajustar hiperparâmetros olhando o teste.
- Usar accuracy com classe rara.
- Grade enorme sem necessidade.
- Buscar fora do Pipeline (vaza).
- Reportar o `best_score_` como se fosse o teste.

Boas práticas

- Ajuste na validação; confirme no teste.
- Escolha o scoring da métrica que importa.
- Comece com uma grade pequena e refine.
- Sempre GridSearchCV sobre o Pipeline.
- Reporte média $\pm$ desvio e o resultado do teste.

Na próxima (Aula 09): quando a grade explode, a busca aleatória e o AutoML acham bons ajustes sem varrer tudo.